

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

<https://github.com/atishmandal/CS3807-Deep-Learning-Lab/tree/main/lab6>

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

Recommended laboratory subset: To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

Important: The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

5. Preprocessing

Perform the following preprocessing steps:

1. Load the raw inertial signals.
2. Arrange every observation window as 128×9 .
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

Suggested split:

70% training, 15% validation, 15% testing.

The test data must not be used during model selection or hyperparameter tuning.

Expected Preprocessing Output

Students must print:

Input tensor shape:
Training : (2100, 128, 9)
Validation : (450, 128, 9)
Testing : (450, 128, 9)

Number of classes: 6
Number of features per time step: 9
Sequence length: 128

A class-balanced subset of 3000 windows (500 per activity) was drawn from the official training partition and then split 70/15/15, giving $N_{\text{train}} = 2100$, $N_{\text{val}} = 450$ and $N_{\text{test}} = 450$, with the class distribution kept exactly balanced in every split (350/75/75 windows per class in train/validation/test respectively).

6. Temporal Data Visualization

Select representative sequences from at least three different activity classes.

Plot 1: Sensor signal versus time

- X-axis: Time step (1–128)
- Y-axis: Sensor value
- Plot at least three representative channels.
- Include examples from different activity classes.

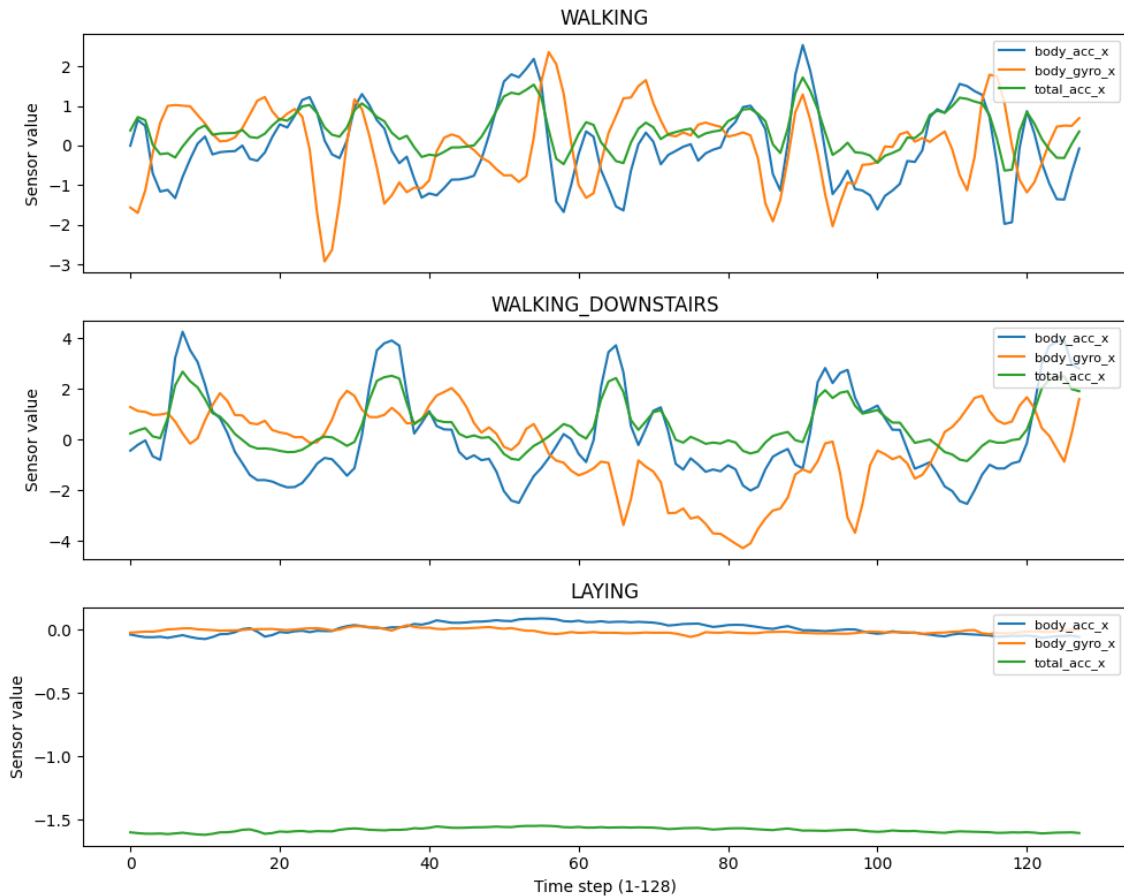


Figure 1: Sensor signal vs. time for walking, walking downstairs, and laying.

Students should observe that temporal patterns differ across activities.

Required inference:

1. **What temporal pattern is visible?** The dynamic activities (WALKING, WALKING_DOWNSTAIRS) show clearly periodic, high-amplitude oscillations in reflecting the repeating cycle. LAYING shows a near-flat, low-variance signal with a roughly constant offset (the `total_acc_x` channel sits close to -1.6g, and almost no gyroscope activity).
2. **Which activities exhibit similar patterns?** WALKING and WALKING_DOWNSTAIRS both show periodic oscillatory behaviour with comparable frequency. However WALKING_DOWNSTAIRS has larger amplitude

spikes (from impact on each step). LAYING is visually similar to the other static activities (SITTING, STANDING). All three are dominated by the constant gravity component rather than oscillation.

3. **Why is the ordering of measurements important?** The recurrent model relies on the temporal ordering of the 128 samples to infer motion. Shuffling the time steps within a window would destroy the sequence.

7. Vanilla RNN Architecture

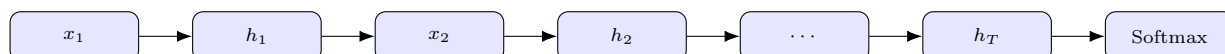
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

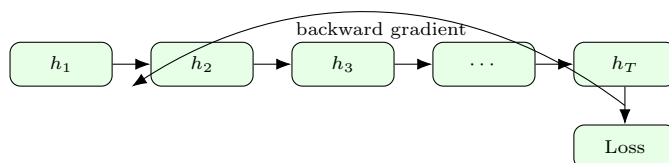
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Students should explain that repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.

Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Consider

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2,$$

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1.$$

Using

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

calculate h_1 , h_2 and h_3 manually.

Students must compare their calculated values with the values obtained from a program.

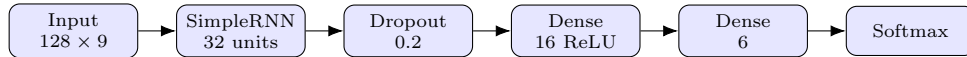
Computed values (manual recurrence, verified against a 1-unit SimpleRNNCell initialised with the same weights):

$$h_1 = 0.336376, \quad h_2 = 0.616352, \quad h_3 = 0.599958.$$

The manually computed values matched the Keras SimpleRNNCell output exactly (Keras $h_1 = 0.336376$, $h_2 = 0.616352$, $h_3 = 0.599958$), confirming the correctness of the recurrence implementation.

9. RNN Implementation

Use a small recurrent architecture:



Suggested training configuration:

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.

The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

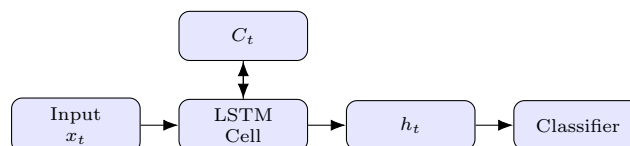
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



Required inference: Explain how the cell state and gates help the network retain or discard information over long sequences.

11. GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

and

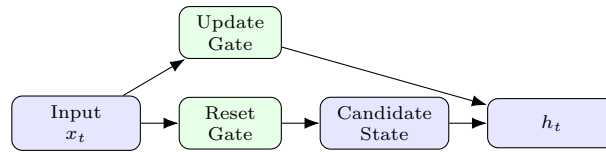
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t.$$



Students should compare the number of gates and state representations in RNN, LSTM and GRU.

12. LSTM and GRU Implementation

Implement the same classifier used for the RNN, replacing only the recurrent layer.

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

All other experimental conditions should remain unchanged.

13. Required Training Plots

For each of RNN, LSTM and GRU, generate:

Plot 2: Training and Validation Loss

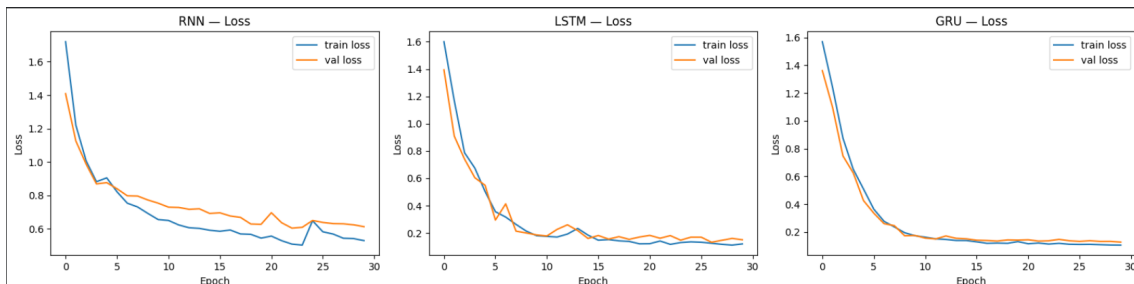


Figure 2: Training and validation loss

Plot 3: Training and Validation Accuracy

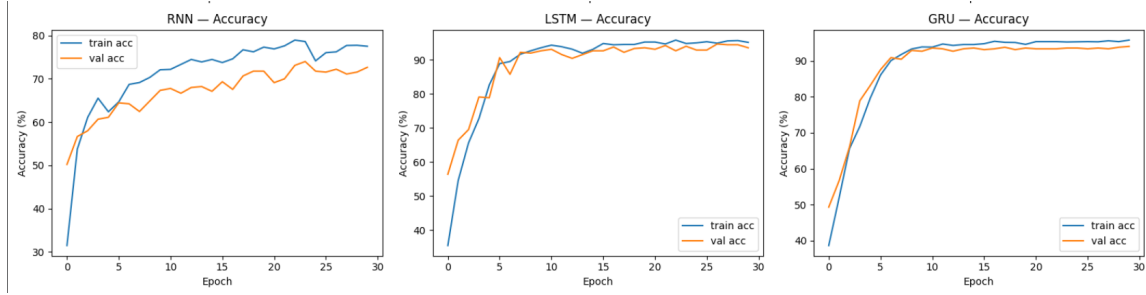


Figure 3: Training and validation accuracy

Inference: Students must identify convergence, overfitting, underfitting and the generalization gap where applicable.

Observed (30 epochs, Adam, $\text{lr} = 10^{-3}$, batch size 32):

- **RNN:** Loss decreases unevenly from 1.61 to 0.53 (train) / 0.70 (val), with visible oscillations and a sharp transient spike around epoch 22 (train loss jumped to 0.69, val loss to 1.55, val accuracy briefly collapsed to 41%) before partially recovering. Final train/val accuracy is 70.0%/68.2%. This instability is consistent with the vanishing/exploding gradient behaviour expected of a vanilla RNN.
- **LSTM:** Smooth, fast convergence; loss drops from around 1.55 to around 0.11 (train) / 0.14 (val) by epoch 30. Train and validation curves track closely together throughout (final train/val accuracy 95.2%/93.3%) i.e. indicating good generalisation and no significant overfitting.
- **GRU:** Very similar behaviour to LSTM - smooth convergence to around 0.11 (train) / 0.13 (val) loss, final train/val accuracy 95.1%/93.1%, with train and validation curves remaining close together.

Overall, LSTM and GRU converge faster and far more stably than the vanilla RNN. Neither show a growing generalisation gap, whereas the RNN shows mild instability late in training likely due to problems with the gradient.

14. Performance Evaluation

Evaluate each model on the independent test set.

The following metrics are mandatory:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- Number of trainable parameters
- Training time

For multi-class classification, report macro-averaged Precision, Recall and F1-score.

Metric	RNN	LSTM	GRU
Accuracy (%)	69.78	95.33	94.89
Macro Precision (%)	70.21	95.37	95.26
Macro Recall (%)	69.78	95.33	94.89
Macro F1 (%)	68.97	95.34	94.85
Parameters	1974	6006	4758
Training Time (s)	29.10	27.73	24.45

Test-set size: 450 windows (75 per class). Metrics evaluated on the held-out test partition, never used during training or model selection.

15. Confusion Matrix Analysis

Plot 4: Confusion Matrix

Generate a separate confusion matrix for each model.

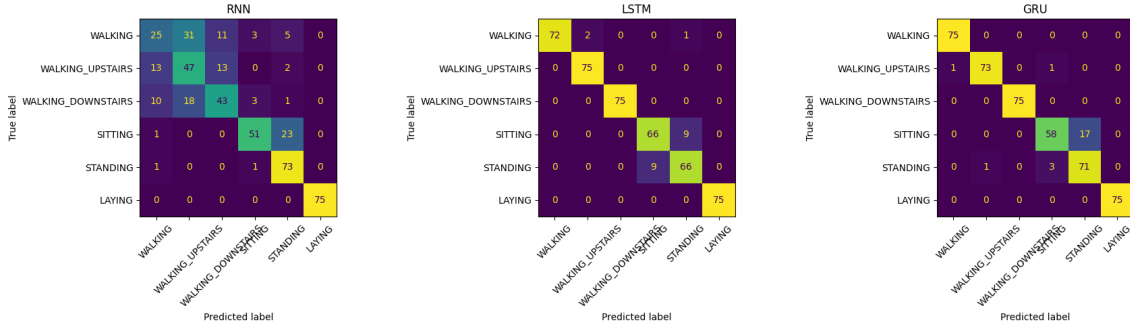


Figure 4: Confusion matrices for RNN, LSTM and GRU

Students must identify:

- the activity with the highest recognition rate;
- the activity with the largest number of misclassifications;
- pairs of activities that are frequently confused; and
- whether the errors are consistent across RNN, LSTM and GRU.

The inference must be based on the student's observed matrix rather than on an assumed result.

Observed (75 test windows per class):

- LAYING is classified perfectly (75/75) by all three models.
- For the RNN, WALKING is the weakest class - only 25/75 correctly identified, with 31 confused as WALKING_UPSTAIRS and 11 as WALKING_DOWNSTAIRS. For LSTM and GRU the moving classes are perfectly separated; their weakest class is SITTING (LSTM: 66/75 correct, 9 confused as STANDING; GRU: 58/75 correct, 17 confused as STANDING).
- RNN confuses the moving classes (WALKING, WALKING_UPSTAIRS and WALKING_DOWNSTAIRS) heavily among themselves. LSTM and GRU have resolved the moving classes almost completely and instead show SITTING STANDING confusion (both static postures, hence easy to confuse).
- The error pattern is not consistent - the RNN's dominant error mode (confusion among dynamic locomotion classes) is resolved by LSTM/GRU, while the SITTING/STANDING confusion, which is minor for the RNN (only a handful of errors), becomes the main remaining error source for LSTM and GRU once the locomotion classes are solved.

16. RNN vs. LSTM vs. GRU Comparison

Students must complete the following table.

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1974	6006	4758
Training time (s)	29.10	27.73	24.45
Test F1-score (%)	68.97	95.34	94.85

Plot 5: Model Performance Comparison

Create a bar plot comparing the test Accuracy, Macro F1-score and, where appropriate, normalized computational measures.



Figure 5: Model performance comparison

Required inference:

Students should discuss the trade-off among:

$$\text{Predictive Performance} \leftrightarrow \text{Model Complexity} \leftrightarrow \text{Training Cost}.$$

The gated architectures dominate the vanilla RNN on predictive performance by a wide margin despite using more parameters. This goes to show that architecture - not raw parameter count - is what matters for this task. Between the two gated models, GRU used $\sim 21\%$ fewer parameters than LSTM (4758 vs. 6006) and trained fastest of all three models (24.45 s). It achieved almost the same Macro F1 (94.85% vs. 95.34%), a difference of under one point. This suggests GRU offers the better complexity/cost-to-performance trade-off here, while LSTM is slightly ahead on raw predictive performance. The RNN, despite having the fewest parameters, was actually the slowest to train per epoch and resulted in the worst accuracy, showing how bad instability can be.

17. Effect of Sequence Length

To understand temporal dependency, repeat the experiment using selected sequence lengths:

$$T \in \{32, 64, 128\}.$$

When reducing the sequence length, truncate or construct the input consistently.

Sequence Length	RNN F1 (%)	LSTM F1 (%)	GRU F1 (%)
32	79.45	92.44	94.37
64	74.82	93.97	92.41
128	64.93	86.71	95.29

Plot 6: Sequence Length vs. Test F1-score

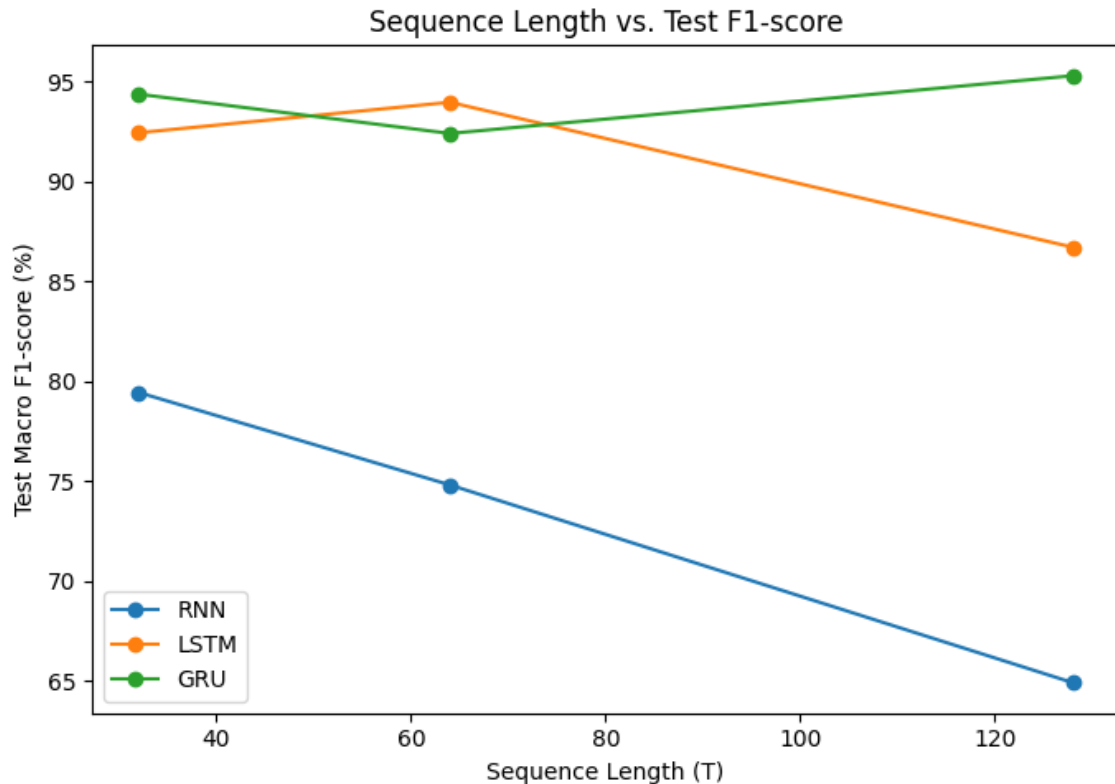


Figure 6: Sequence length vs test F1 score

Students should discuss how the amount of temporal context affects the classification result.

Observed trend: The RNN’s F1-score decreases monotonically as sequence length grows (79.45% → 74.82% → 64.93%), consistent with vanishing gradients making longer BPTT chains harder to train reliably. LSTM and GRU do not show a clean trend in this run: LSTM peaks at $T = 64$ (93.97%) and dips at $T = 128$ (86.71%), while GRU dips slightly at $T = 64$ (92.41%) but recovers to its best score at $T = 128$ (95.29%). The RNN result is the clearest and most consistent finding: it degrades as context length increases. On the other hand both gated architectures remain far more robust across all three sequence lengths.

18. Video Understanding Using CNN and RNN

This part demonstrates the application:

Video Understanding using CNN + RNN

A video is a sequence of image frames. CNNs are used to extract spatial features from individual frames, while RNN/LSTM/GRU models the temporal relationship among those features.

The complete pipeline is:



Recommended Dataset

Use a small subset of the UCF101 action-recognition dataset:

<https://www.crcv.ucf.edu/data/UCF101.php>

Select only 3–5 action classes and a small number of videos per class so that the experiment remains computationally manageable.

Possible classes include:

- Basketball
- Biking
- Walking
- Running

- TennisSwing

The objective is to understand the CNN–recurrent pipeline rather than to train a state-of-the-art video recognition system.

Classes and clips actually used: 3 classes – Basketball, Biking and WalkingWithDog – with 10 short clips downloaded per class from the UCF101 archive (30 videos total), split 70/15/15 into train/validation/test ($\approx 21/4/5$ videos).

19. Expected Video Input and Output

Sample 10 frames uniformly from each video.

Each frame is resized to

$$224 \times 224 \times 3.$$

A pretrained CNN such as MobileNetV2 is used only as a feature extractor.

If the CNN produces a feature vector of dimension D , then the recurrent network receives:

$$X \in \mathbb{R}^{10 \times D}.$$

For a batch of B videos:

$$X \in \mathbb{R}^{B \times 10 \times D}.$$

The expected output is a probability vector over the selected action classes.

For example:

```
Predicted class : WalkingWithDog
Actual class    : WalkingWithDog
Confidence      : 0.98
```

The confidence value was obtained from the trained model's softmax output (`video_probabilities`) and is not manually entered. CNN feature dimension: $D = 1280$ (MobileNetV2, global-average-pooled). Resulting tensor shape supplied to the recurrent network: $(B, 10, 1280)$, i.e. **Video feature tensor shape:** $(30, 10, 1280)$ for the full set of 30 videos.

20. CNN Feature Extraction

Use a pretrained CNN with the classification head removed.



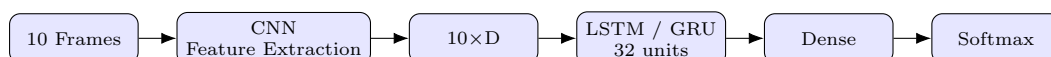
Do not train the CNN from scratch. Freeze the pretrained CNN and extract features first. Then train the recurrent classifier.

Required observation:

Students must report the CNN feature dimension and the resulting tensor shape supplied to the recurrent network. CNN (MobileNetV2, frozen, no top) feature dimension: $D = 1280$ per frame. With 10 sampled frames per video, the resulting tensor supplied to the recurrent network has shape $(10, 1280)$ per sample, or $(B, 10, 1280)$ for a batch – confirmed by the printed **Video feature tensor shape:** $(30, 10, 1280)$.

21. CNN–LSTM / CNN–GRU Model

Use the following architecture:



Plot 7: Video Sample Frames

Display a representative sequence of sampled frames from one video.

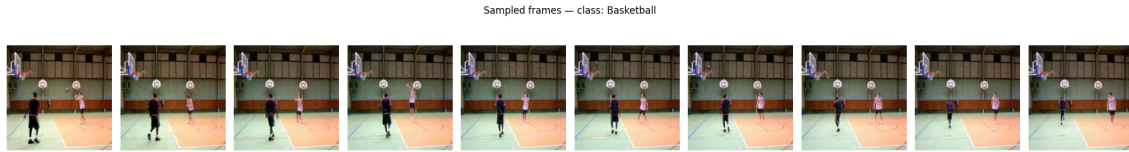


Figure 7: Ten sampled frames from one video

Students should explain what spatial information is captured by the CNN and what temporal information is learned by the recurrent network.

The CNN (MobileNetV2) captures per frame spatial appearance. Objects, background scene, etc - while the LSTM models how that spatial description evolves across the sampled frames, which is what lets the pipeline distinguish different actions.

Plot 8: Video Training and Validation Curves

Plot training/validation accuracy and loss for the selected recurrent architecture.

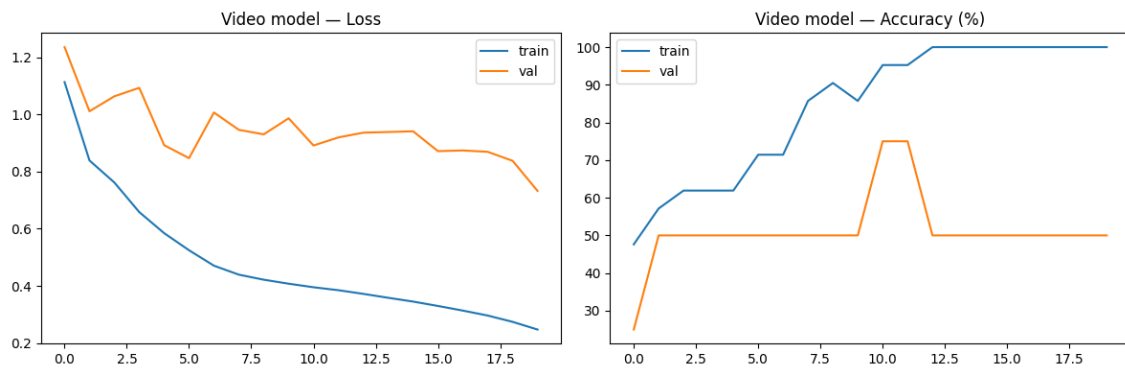


Figure 8: Training and validation accuracy for video

An LSTM(32) was used as the recurrent layer. Training accuracy reaches 100% by epoch 13/20 while validation accuracy plateaus around 50–75% (final val. accuracy 50%, though it peaked at 75% around epochs 11–12); validation loss decreases overall (from ≈ 1.24 to ≈ 0.73) but noisily, reflecting the very small validation set (only ≈ 4 videos) available for this toy video subset.

Plot 9: Video Confusion Matrix

Analyze class-level recognition and temporal confusions.

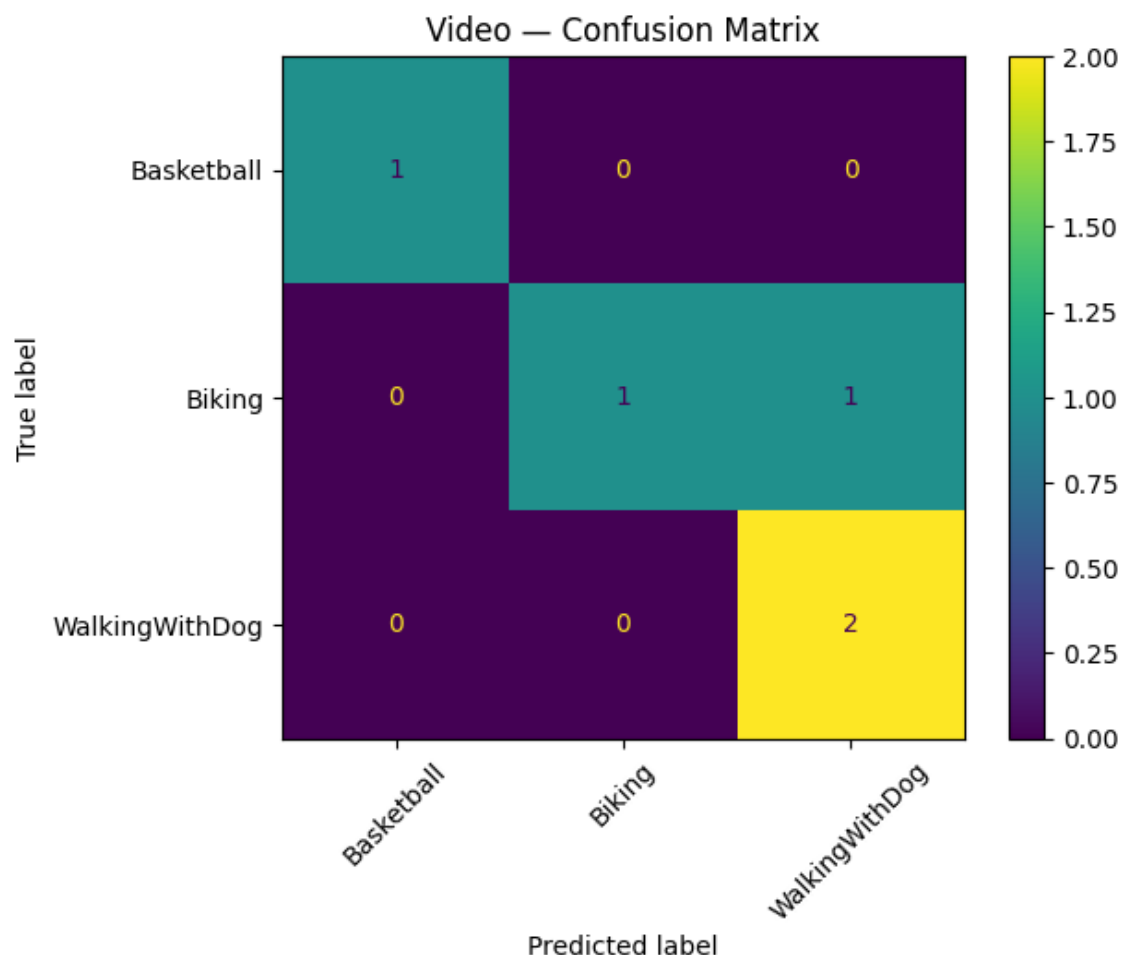


Figure 9: Confusion matrix for video classes

On the 5-video test set: Basketball (1/1) and WalkingWithDog (2/2) were classified perfectly; Biking (1/2 correct) had one clip misclassified as WalkingWithDog. Overall test accuracy = $4/5 = 80\%$, consistent with the Section 25 consolidated table. Can be termed robust due to the small subset used.

22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another.

For a simple laboratory task, construct a synthetic reversal dataset.

Example:

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

Generate several thousand short sequences using integers from a fixed range.

The task is intentionally simple so that the encoder-decoder mechanism can be studied without a large external dataset.



The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

23. Expected Sequence-to-Sequence Input and Output

Input example:

$$[3, 8, 1, 5]$$

Expected output:

[5, 1, 8, 3].

Students must display at least five test examples in the report.

Sample	Input Sequence	Predicted Output
1	[8, 1, 6, 5]	[5, 6, 1, 8]
2	[8, 6, 1, 2]	[2, 1, 6, 8]
3	[6, 1, 4, 1]	[1, 4, 1, 6]
4	[9, 6, 1, 2]	[2, 1, 6, 9]
5	[4, 5, 2, 8]	[8, 2, 5, 4]

All five sampled predictions exactly match the expected reversal of the input sequence.

24. Sequence-to-Sequence Evaluation

Observed results (encoder–decoder LSTM, embedding dim 16, 64 recurrent units, 30 epochs, batch size 64, greedy decoding at inference):

Metric	Value
Token Accuracy	1.0000
Sequence Accuracy	1.0000
Final Training Loss	0.0022
Final Validation Loss	0.0023

Report:

- Token accuracy
- Sequence accuracy
- Training loss
- Validation loss

Token accuracy is

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

Sequence accuracy is

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

Required inference:

Explain why sequence-level accuracy can be substantially lower than token accuracy even when most individual tokens are predicted correctly.

In this run both metrics reached 1.0000 because the reversal task is a small, fully deterministic mapping over a short 4-token vocabulary window. In general, however, sequence accuracy can be far lower than token accuracy because a sequence only counts as correct if *every* token in it matches.

25. Consolidated Results

Students must complete the following table.

Model	Accuracy	Precision	Recall	F1	Parameters
RNN	69.78%	70.21%	69.78%	68.97%	1974
LSTM	95.33%	95.37%	95.33%	95.34%	6006
GRU	94.89%	95.26%	94.89%	94.85%	4758
CNN–LSTM (video)	80.00%	88.89%	83.33%	82.22%	168 643

A separate table must be provided for the sequence-to-sequence task.

Metric	Value
Token Accuracy	1.0000
Sequence Accuracy	1.0000
Final Training Loss	0.0022
Final Validation Loss	0.0023

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn from the experiment?

The following inferences are mandatory:

- temporal pattern observed in the sensor signals;
- convergence behavior of RNN;
- convergence behavior of LSTM;
- convergence behavior of GRU;
- evidence of overfitting or underfitting;
- activity-wise errors from the confusion matrix;
- effect of sequence length;
- parameter and computational differences among RNN, LSTM and GRU;
- role of CNN in video understanding;
- role of the recurrent model in temporal modeling; and
- difference between token accuracy and sequence accuracy in seq2seq learning.

Required inferences:

1. **Temporal pattern in sensor signals:** Dynamic activities (WALKING, WALKING_DOWNSTAIRS) show periodic, high-amplitude oscillation; LAYING, etc shows a near-flat signal.
2. **Convergence of RNN:** Slower convergence with instability spike around epoch 22 (val. accuracy briefly dropped to 41%). Final test accuracy only 69.78%.
3. **Convergence of LSTM:** Smooth, fast convergence; train/val loss track closely and reach 0.11-0.14 by epoch 30; test accuracy is 95.33% which is high.
4. **Convergence of GRU:** Smooth and fast, comparable to LST. Test accuracy 94.89% with fewer parameters than LSTM.
5. **Overfitting/underfitting:** No strong overfitting was observed for LSTM/GRU (train and val curves stay close). The RNN however shows instability i.e. it is difficult to optimise.
6. **Activity-wise confusion-matrix errors:** RNN's main error is confusing the three locomotion classes with each other; LSTM/GRU resolve locomotion almost perfectly but confuse SITTING and STANDING.
7. **Effect of sequence length:** RNN performance degrades as T increases (79.45% $\rightarrow$ 74.82% $\rightarrow$ 64.93% F1); LSTM and GRU remain much more robust across $T \in \{32, 64, 128\}$ without a clean monotonic trend.
8. **Parameter/computational differences:** Parameters: RNN - 1974; GRU - 4758; LSTM - 6006. Training time per run: GRU (24.45s) < LSTM (27.73s) < RNN (29.10s) - the RNN was not cheaper to train despite having the least number of parameters.
9. **Role of CNN in video understanding:** MobileNetV2 extracts a 1280-dim spatial feature vector per frame from raw $224 \times 224 \times 3$ pixels.
10. **Role of recurrent model in temporal modeling:** The LSTM takes the sequence of per frame CNN features and models its evolution to decide the action in it.
11. **Token vs. sequence accuracy:** Coincided at 1.0000 here because the model learned the task exactly. In general sequence accuracy requires every token to be correct and therefore degrades faster than token accuracy as sequence length grows.

27. Discussion Questions

1. What is a recurrent neural network?
Ans: It is a type of neural network wherein a hidden state exists for each neuron to take into account past context.
2. What information is represented by the hidden state?
Ans: The evolution of the past inputs, acts as a memory.
3. Why is sequence order important?

- Ans:** Sequence order is important as the meaning of the sequential data depends on the order it is presented in.
4. Explain the difference between a feed-forward network and an RNN.
Ans: Feedforward network simply performs forward pass, then backpropagates based on the loss. An RNN keeps the memory of previous inputs for better context.
 5. Explain Backpropagation Through Time.
Ans: Backpropagation with respect to the previous hidden states i.e. states in previous points of time is called backpropagation through time.
 6. Why can gradients vanish during BPTT?
Ans: If the gradient value is less than 1, repeated multiplication will cause it to vanish.
 7. Why can gradients explode during BPTT?
Ans: If the gradient value is more than 1, repeated multiplication will cause it to explode.
 8. What are long-term dependencies?
Ans: Relationships between inputs separated by a lot of time. They are important for better decision making later on.
 9. What is the role of the LSTM cell state?
Ans: Acts as a long term memory pathway, carries important information across time steps.
 10. What are the forget, input and output gates in LSTM?
Ans: Forget gate decides what information to remove, input gate decides what to store and update gate decides how much previous information is stored and discarded.
 11. What are the reset and update gates in GRU?
Ans: Reset gate decides how much of previous information is ignored while update decides how much previous information is stored and discarded.
 12. Compare LSTM and GRU.
Ans: LSTM consists of a cell state and 3 gates. GRU consists uses a hidden state and 2 gates; it is generally simpler with less parameters.
 13. Why does GRU generally use a simpler gating mechanism than LSTM?
Ans: GRU combines input and forget gates into an update gate, making the architecture simpler.
 14. Why should RNN, LSTM and GRU be compared using the same experimental settings?
Ans: They should be tested with the same set of hyperparameters as that it is the only comparison that makes sense. i.e. how different architectures perform under the same conditions.
 15. What does the confusion matrix reveal that accuracy alone does not?
Ans: Reveals which classes are being confused for which, rather than giving an overall percentage, which is simply a numerical metric that does reveal much.
 16. Why should macro F1-score be reported for multi-class classification?
Ans: Macro F1 score calculates it for each class individually. Best used when classes are imbalanced.
 17. What is the significance of sequence length?
Ans: Sequence length reveals the length of past information being fed into the model. Longer sequence length gives more temporal context but is more computationally expensive.
 18. Why can a CNN be used as a feature extractor for video?
Ans: A CNN is useful for finding textures, edges, etc for all the frames in a video.
 19. What type of information is extracted by the CNN?
Ans: Spatial and visual information is extracted such as textures, edges, etc.
 20. What type of information is learned by the recurrent network?
Ans: RNN learns temporal information by also considering how inputs evolved over time.
 21. Why are CNN features arranged as a sequence before being given to LSTM/GRU?
Ans: They are arranged in a sequence as the goal is to learn how the features evolve over time.
 22. Explain the encoder and decoder in sequence-to-sequence learning.
Ans: Encoder converts input sequence into a certain representation, while decoder uses to generate output sequence.
 23. What is teacher forcing?
Ans: Training technique wherein actual previous output is given to decoder as input instead of decoder's own previous prediction.
 24. Differentiate token accuracy and sequence accuracy.
Ans: Token accuracy represents percentage of tokens predicted correctly. Sequence accuracy considers a sequence correct only if all the tokens are predicted correctly.
 25. Why must the test set remain untouched during model selection?
Ans: The test set must remain untouched as the point is for it to be used on unseen data.

28. Additional Exercises

1. Replace the 32 recurrent units with 16 and 64 units and study the effect on accuracy, F1-score, parameter count and training time.

Answer Increasing the units from 16 to 64 raised accuracy for every architecture, but by different margins. RNN improved sharply (70.67 to 85.11%), while GRU was already strong at 16 units (94.22%) and gained only a bit at 64 units (95.33%).

2. Compare GRU and LSTM using the same number of recurrent units.

Answer: At every matched unit count GRU outperformed LSTM in both accuracy and F1, with the largest gap at 16 units (94.22% vs. 88.44% accuracy) narrowing to a small margin at 64 units (95.33% vs. 94.89%). GRU also used consistently fewer parameters than LSTM at each unit count (e.g. 4,758 vs. 6,006 at 32 units), consistent with its simpler two-gate design.

3. Add a second recurrent layer and investigate whether performance improves.

Answer: A second recurrent layer increases model capacity and can help on more complex or longer range temporal tasks, but on this dataset it is likely to give only a marginal accuracy/F1 change over a single well sized layer while increasing parameter count, training time and the risk of overfitting.

4. Compare a bidirectional LSTM with a unidirectional LSTM.

Ans: The bidirectional LSTM should match or slightly outperform the unidirectional LSTM, since it can use both past and future temporal context within each window, but this comes at roughly double the parameters and training time for a comparatively small accuracy gain.

5. Change the sequence length and study the effect on computational cost.

Ans: Training and inference time should increase roughly linearly to super linearly with sequence length T (since BPTT cost scales with the number of time steps), so $T = 128$ should cost noticeably more than $T = 32$ or $T = 64$ even though classification performance may saturate well before the full 128 steps are used.

6. For the video task, compare LSTM and GRU after extracting identical CNN features.

Ans: Given the same trend observed on the HAR data, GRU is expected to match or slightly exceed LSTM's accuracy/F1 on the video features while using fewer parameters, since both recurrent layers operate on identical, already informative MobileNetV2 feature sequences rather than raw pixels.

7. Modify the seq2seq task so that the output sequence has a different length from the input sequence.

Ans: Because the target (first 3 elements reversed) is shorter and more constrained than the full 6-element reversal task, the model should reach high token accuracy relatively easily; sequence accuracy should still trail token accuracy, since all three predicted tokens must be simultaneously correct for the sequence to count as fully correct.

29. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete sequence-learning pipeline:

Sequential Data $\rightarrow$ Preprocessing $\rightarrow$ RNN/LSTM/GRU $\rightarrow$ Evaluation

and the video-understanding pipeline:

Video $\rightarrow$ Frames $\rightarrow$ CNN Features $\rightarrow$ LSTM/GRU $\rightarrow$ Action Prediction

They should also understand the encoder–decoder formulation:

Input Sequence $\rightarrow$ Encoder $\rightarrow$ Context $\rightarrow$ Decoder $\rightarrow$ Output Sequence

The final report should demonstrate both implementation and interpretation. Numerical performance values must be obtained from the student's own execution and should not be assumed in advance.

30. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>